
Proyecto de Ejemplo: *User Quality Demo*

Aplicación Java + Pruebas Unitarias, Integración y Aceptación (BDD)

Curso: Estándares y Métricas de Calidad

Semana: 8 – Testing y Aseguramiento de Calidad

Profesor: Cesar Alfonso

Índice

1. Descripción del proyecto
 2. Estructura del proyecto
 3. Archivo `pom.xml`
 4. Código fuente de la aplicación
 5. Pruebas unitarias
 6. Pruebas de integración
 7. Pruebas de aceptación (BDD)
 8. Tabla comparativa
-

Descripción del proyecto

Este proyecto demuestra cómo implementar **tres niveles de pruebas** en Java:

- **Unitarias:** validan reglas de negocio con mocks.
- **Integración:** validan interacción real entre componentes.
- **Aceptación (BDD):** validan criterios de usuario con escenarios Given–When–Then.

La aplicación es un pequeño servicio de **gestión de usuarios** con operaciones CRUD y reglas de negocio como:

- El email debe ser único.
 - El nombre es obligatorio.
-

Estructura del proyecto

```
user-quality-demo/  
├─ pom.xml
```

```
└─ src
   └─ main/java/com/example/usermanagement
      ├── model/User.java
      ├── repository/UserRepository.java
      ├── repository/InMemoryUserRepository.java
      └── service/UserService.java
   └─ test/java/com/example/usermanagement
      ├── unit/UserServiceUnitTest.java
      ├── integration/UserServiceIntegrationTest.java
      └── acceptance/UserServiceAcceptanceTest.java
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>user-quality-demo</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <junit.jupiter.version>5.10.0</junit.jupiter.version>
    <mockito.version>5.11.0</mockito.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>${junit.jupiter.version}</version>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>${mockito.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
```

```
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
        <configuration>
            <useModulePath>false</useModulePath>
        </configuration>
    </plugin>
</plugins>
</build>
</project>
```

Código fuente de la aplicación

User.java

```
package com.example.usermanagement.model;

/**
 * Entidad User con reglas simples:
 * - id: identificador único
 * - name: obligatorio
 * - email: obligatorio y único
 */
public class User {

    private Long id;
    private String name;
    private String email;

    public User(Long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public User(String name, String email) {
        this(null, name, email);
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

UserRepository.java

```
package com.example.usermanagement.repository;

import com.example.usermanagement.model.User;
import java.util.List;
import java.util.Optional;

/**
 * Contrato del repositorio.
 * Se mockea en pruebas unitarias.
 * Se implementa en memoria para pruebas de integración.
 */
public interface UserRepository {

    User save(User user);

    Optional<User> findById(Long id);

    Optional<User> findByEmail(String email);

    List<User> findAll();

    void deleteById(Long id);
}
```

InMemoryUserRepository.java

```
package com.example.usermanagement.repository;

import com.example.usermanagement.model.User;
import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Implementación en memoria para pruebas de integración.
 */
public class InMemoryUserRepository implements UserRepository {

    private final Map<Long, User> storage = new HashMap<>();
    private final AtomicLong idSequence = new AtomicLong(1);

    @Override
    public User save(User user) {
        if (user.getId() == null) {
            user.setId(idSequence.getAndIncrement());
        }
        storage.put(user.getId(), user);
    }
}
```

```
        return user;
    }

    @Override
    public Optional<User> findById(Long id) {
        return Optional.ofNullable(storage.get(id));
    }

    @Override
    public Optional<User> findByEmail(String email) {
        return storage.values().stream()
            .filter(u -> u.getEmail().equalsIgnoreCase(email))
            .findFirst();
    }

    @Override
    public List<User> findAll() {
        return new ArrayList<>(storage.values());
    }

    @Override
    public void deleteById(Long id) {
        storage.remove(id);
    }
}
```

UserService.java

```
package com.example.usermanagement.service;

import com.example.usermanagement.model.User;
import com.example.usermanagement.repository.UserRepository;
import java.util.List;

/**
 * Servicio con reglas de negocio:
 * - nombre obligatorio
 * - email obligatorio y único
 */
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User createUser(String name, String email) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("El nombre es obligatorio");
    }
}
```

```
        if (email == null || email.isBlank())
            throw new IllegalArgumentException("El email es obligatorio");

        userRepository.findByEmail(email).ifPresent(u -> {
            throw new IllegalArgumentException("El email ya está registrado");
        });

        return userRepository.save(new User(name, email));
    }

    public User updateUser(Long id, String name, String email) {
        User existing = userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Usuario no
encontrado"));

        if (name != null) {
            if (name.isBlank())
                throw new IllegalArgumentException("El nombre no puede estar
vacío");
            existing.setName(name);
        }

        if (email != null) {
            if (email.isBlank())
                throw new IllegalArgumentException("El email no puede estar
vacío");

            userRepository.findByEmail(email).ifPresent(other -> {
                if (!other.getId().equals(id))
                    throw new IllegalArgumentException("El email ya está
registrado por otro usuario");
            });

            existing.setEmail(email);
        }

        return userRepository.save(existing);
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }
}
```

Pruebas unitarias

Propósito

Validar reglas de negocio del servicio **sin usar repositorio real**, usando **mocks**.

Herramientas

- JUnit 5
- Mockito

Ejecución

```
mvn test
```

UserServiceUnitTest.java

```
package com.example.usermanagement.unit;

import com.example.usermanagement.model.User;
import com.example.usermanagement.repository.UserRepository;
import com.example.usermanagement.service.UserService;
import org.junit.jupiter.api.*;
import org.mockito.Mockito;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

/**
 * Pruebas unitarias con mocks.
 */
public class UserServiceUnitTest {

    private UserRepository repo;
    private UserService service;

    @BeforeEach
    void setup() {
        repo = Mockito.mock(UserRepository.class);
        service = new UserService(repo);
    }

    @Test
    void createUser_ok_whenEmailIsUnique() {
        when(repo.findByEmail("cesar@example.com"))
            .thenReturn(Optional.empty());

        when(repo.save(any(User.class)))
            .thenReturn(new User(1L, "Cesar", "cesar@example.com"));
    }
}
```

```
        User u = service.createUser("Cesar", "cesar@example.com");

        assertEquals(1L, u.getId());
        verify(repo, times(1)).save(any(User.class));
    }

    @Test
    void createUser_fails_whenEmailExists() {
        when(repo.findByEmail("ana@example.com"))
            .thenReturn(Optional.of(new User(10L, "Ana", "ana@example.com")));

        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> service.createUser("Ana", "ana@example.com")
        );

        assertEquals("El email ya está registrado", ex.getMessage());
        verify(repo, never()).save(any(User.class));
    }

    @Test
    void updateUser_fails_whenUserNotFound() {
        when(repo.findById(99L)).thenReturn(Optional.empty());

        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> service.updateUser(99L, "Nuevo", null)
        );

        assertEquals("Usuario no encontrado", ex.getMessage());
    }
}
```

Pruebas de integración

Propósito

Validar interacción real entre **UserService** + **InMemoryUserRepository**.

Herramientas

- JUnit 5
- Repositorio real en memoria

UserServiceIntegrationTest.java


```
package com.example.usermanagement.integration;

import com.example.usermanagement.model.User;
import com.example.usermanagement.repository.*;
import com.example.usermanagement.service.UserService;
import org.junit.jupiter.api.*;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Pruebas de integración.
 */
public class UserServiceIntegrationTest {

    private UserRepository repo;
    private UserService service;

    @BeforeEach
    void setup() {
        repo = new InMemoryUserRepository();
        service = new UserService(repo);
    }

    @Test
    void createAndGetAllUsers_ok() {
        service.createUser("User1", "u1@example.com");
        service.createUser("User2", "u2@example.com");

        List<User> users = service.getAllUsers();

        assertEquals(2, users.size());
    }

    @Test
    void deleteUser_ok() {
        User u = service.createUser("User3", "u3@example.com");
        assertEquals(1, service.getAllUsers().size());

        service.deleteUser(u.getId());
        assertEquals(0, service.getAllUsers().size());
    }
}
```

Pruebas de aceptación (BDD)

Propósito

Validar criterios de usuario mediante escenarios **Given-When-Then**.

Herramientas

- JUnit 5
- Estilo BDD con nombres y comentarios

UserServiceAcceptanceTest.java

```
package com.example.usermanagement.acceptance;

import com.example.usermanagement.repository.*;
import com.example.usermanagement.service.UserService;
import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Pruebas de aceptación estilo BDD.
 */
public class UserServiceAcceptanceTest {

    private UserRepository repo;
    private UserService service;

    @BeforeEach
    void setup() {
        repo = new InMemoryUserRepository();
        service = new UserService(repo);
    }

    @Test
    void givenExistingEmail_whenCreatingUser_thenRejects() {
        // Given
        service.createUser("Existente", "existing@example.com");

        // When
        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> service.createUser("Nuevo", "existing@example.com")
        );

        // Then
        assertEquals("El email ya está registrado", ex.getMessage());
    }

    @Test
    void givenEmptyName_whenCreatingUser_thenRejects() {
        // Given
        String name = " ";
        String email = "noname@example.com";
```

```
// When
IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class,
    () -> service.createUser(name, email)
);

// Then
assertEquals("El nombre es obligatorio", ex.getMessage());
}
```

Tabla comparativa

Tipo de prueba	Alcance	Dependencias	Nivel	Ejemplo
Unitarias	Lógica aislada	Mocks (Mockito)	Bajo	Validar email único
Integración	Componentes reales	Servicio + Repo real	Medio	CRUD en memoria
Aceptación (BDD)	Criterios de usuario	Sistema funcional	Alto	Escenarios Given-When-Then